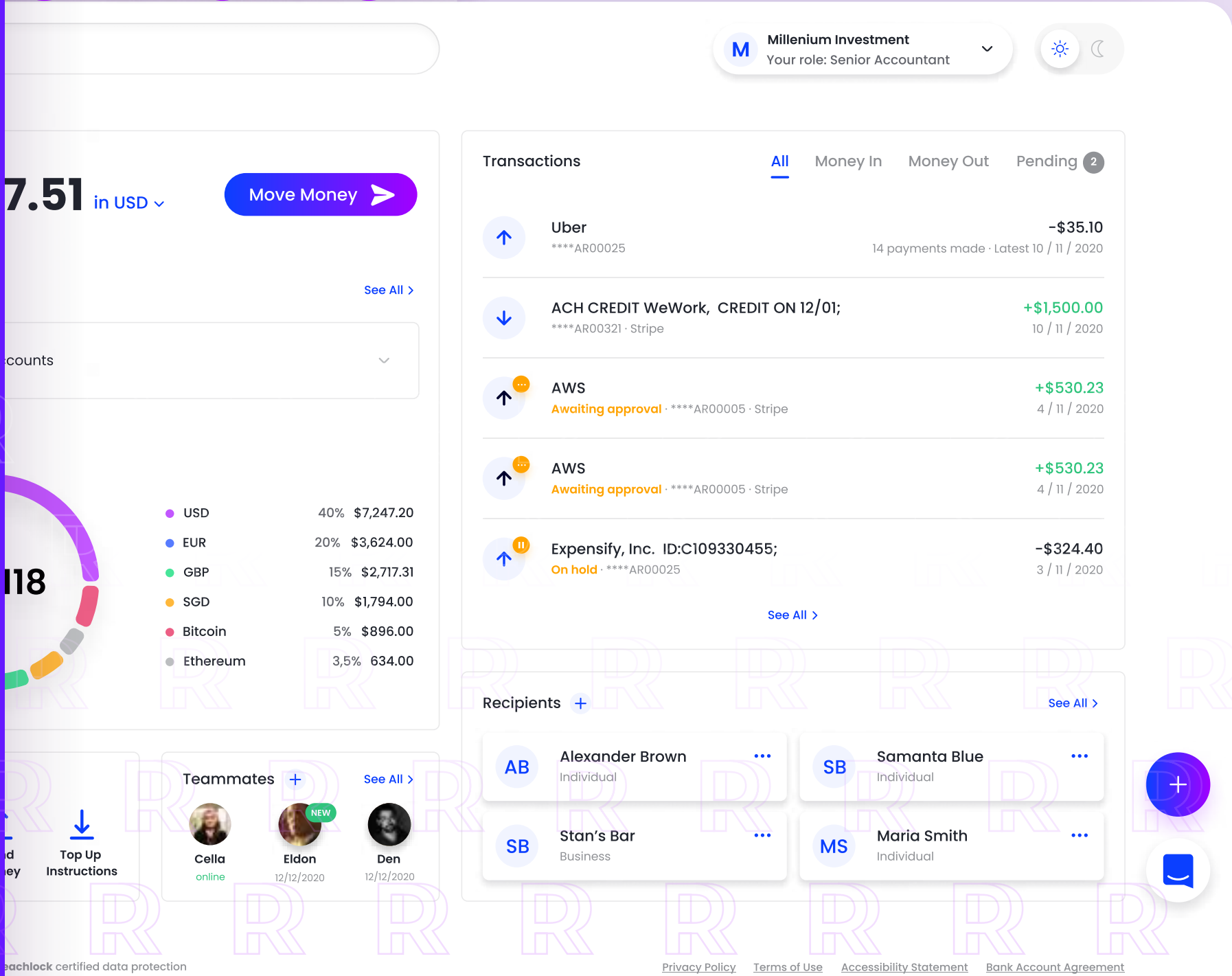
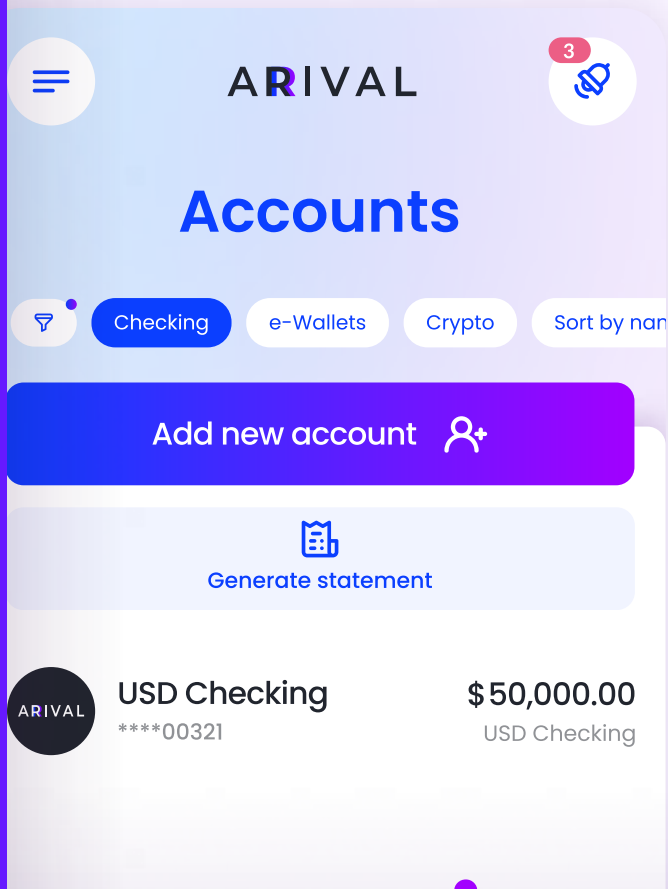


ARIVAL TECH presentation

Building the best project with the best technologies.



[presentation structure]

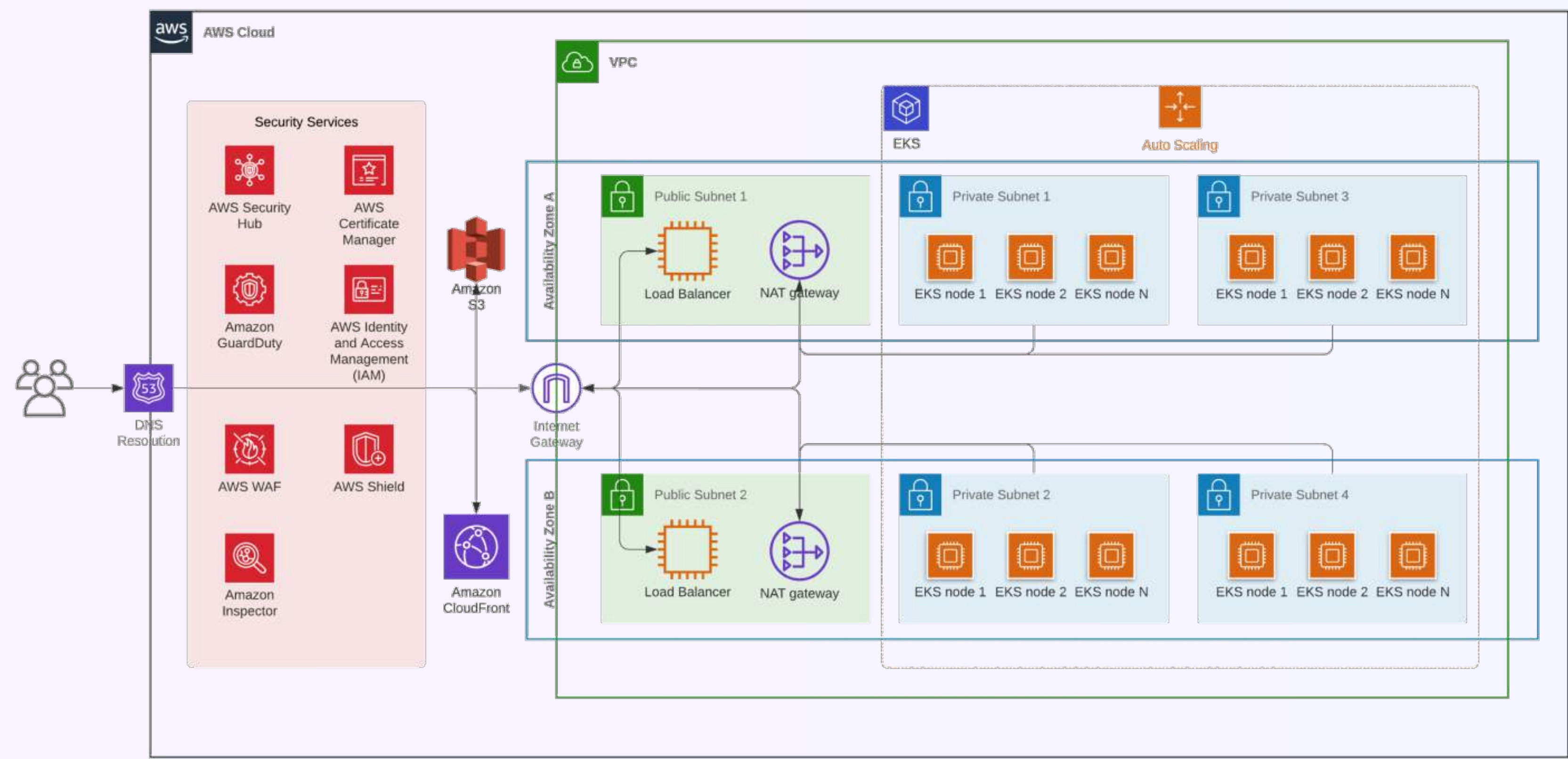
- Ⓜ DevOps section (our cloud infrastructure)
- Ⓜ Bank technical description
- Ⓜ Arival onboarding and compliance solution technical description

[section structure]

- Ⓜ General architecture
- Ⓜ Key modules
- Ⓜ Privacy block
- Ⓜ Security
- Ⓜ Technologies



GENERAL ARCHITECTURE



CORE ELEMENTS OF THE SYSTEM 1

S3 STORAGE

Amazon Simple Storage Service (Amazon S3) is an object storage service that offers industry-leading scalability, data availability, security, and performance. This means customers of all sizes and industries can use it to store and protect any amount of data for a range of use cases, such as data lakes, websites, mobile applications, backup and restore, archive, enterprise applications, IoT devices, and big data analytics. Amazon S3 provides easy-to-use management features so you can organize your data and configure finely-tuned access controls to meet your specific business, organizational, and compliance requirements. Amazon S3 is designed for 99.999999999% (11 9's) of durability, and stores data for millions of applications for companies all around the world.

KUBERNETES

Amazon Elastic Kubernetes Service (Amazon EKS) is a managed container service to run and scale Kubernetes applications in the cloud or on-premises.

EFK

The EFK stack is an acronym used to describe a stack that comprises of three popular projects: Elasticsearch, Fluentd, and Kibana. Often referred to as Elasticsearch, the EFK stack gives you the ability to aggregate logs from all your systems and applications, analyze these logs, and create visualizations for application and infrastructure monitoring, faster troubleshooting, security analytics, and more.

AWS SSM

AWS Systems Manager (formerly known as SSM) is an AWS service that you can use to view and control your infrastructure on AWS. Using the Systems Manager console, you can view operational data from multiple AWS services and automate operational tasks across your AWS resources. Systems Manager helps you maintain security and compliance by scanning your managed instances and reporting on (or taking corrective action on) any policy violations it detects. A managed instance is a machine configured for use with Systems Manager. Systems Manager also helps you configure and maintain your managed instances. Supported machine types include Amazon Elastic Compute Cloud (Amazon EC2) instances, on-premises servers, and virtual machines (VMs), including VMs in other cloud environments.

CLUSTER AUTOSCALER

The Kubernetes Cluster Autoscaler automatically adjusts the number of nodes in your cluster when pods fail or are rescheduled onto other nodes. The Cluster Autoscaler is typically installed as a Deployment in your cluster. It uses leader election to ensure high availability, but scaling is done by only one replica at a time.

AMAZON GUARDDUTY

Amazon GuardDuty is a threat detection service that continuously monitors your AWS accounts and workloads for malicious activity and delivers detailed security findings for visibility and remediation.

CORE ELEMENTS OF THE SYSTEM 2

AWS CLOUDFRONT (CDN)

Amazon CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency, high transfer speeds, all within a developer-friendly environment. CloudFront China has Edge locations in Beijing, Shanghai, Zhongwei, and Shenzhen. These four Edge locations are connected by private network directly to Amazon Web Services China (Beijing) Region operated by Sinnet and Amazon Web Services China (Ningxia) Region operated by NWCD for speedy content delivery to viewers in China. CloudFront works seamlessly with services, including Amazon Shield Standard for DDoS mitigation, and Amazon S3, Elastic Load Balancing, or Amazon EC2 as origins for your applications.

AWS WAF

(AWS WAF is a web application firewall that lets you monitor the HTTP and HTTPS requests that are forwarded to CloudFront, and lets you control access to your content. Based on conditions that you specify, such as the values of query strings or the IP addresses that requests originate from, CloudFront responds to requests either with the requested content or with an HTTP status code 403 (Forbidden). You can also configure CloudFront to return a custom error page when a request is blocked. For more information about AWS WAF, see the AWS WAF Developer Guide.)

HELM / ARGOCD

The Helm package manager for Kubernetes helps install and manage applications on Kubernetes cluster. Argo CD is a declarative, GitOps continuous delivery tool for Kubernetes.

DOCKER

Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker’s methodologies for shipping, testing, and deploying code quickly, you can significantly reduce the delay between writing code and running it in production.

AWS KMS (ENCRYPTION)

AWS Key Management Service (KMS) is an Amazon Web Services product that allows administrators to create, delete and control keys that encrypt data stored in AWS databases and products.

APPLICATION LOAD BALANCER

Elastic Load Balancing automatically distributes your incoming traffic across multiple targets, such as EC2 instances, containers, and IP addresses, in one or more Availability Zones. It monitors the health of its registered targets, and routes traffic only to the healthy targets. Elastic Load Balancing scales your load balancer as your incoming traffic changes over time. It can automatically scale to the vast majority of workloads.

TERRAFORM

Terraform is an open-source infrastructure as code software tool that provides a consistent CLI workflow to manage hundreds of cloud services. Terraform codifies cloud APIs into declarative configuration files.

CORE ELEMENTS OF THE SYSTEM 3

HORIZONTAL POD AUTOSCALER

The Kubernetes Horizontal Pod Autoscaler automatically scales the number of pods in a deployment, replication controller, or replica set based on that resource's CPU utilization. This can help your applications scale out to meet increased demand or scale in when resources are not needed, thus freeing up your nodes for other applications. When you set a target CPU utilization percentage, the Horizontal Pod Autoscaler scales your application in or out to try to meet that target.

PROMETHEUS

Prometheus is an open-source systems monitoring and alerting toolkit originally built at SoundCloud. Since its inception in 2012, many companies and organizations have adopted Prometheus, and the project has a very active developer and user community. It is now a standalone open source project and maintained independently of any company. To emphasize this, and to clarify the project's governance structure, Prometheus joined the Cloud Native Computing Foundation in 2016 as the second hosted project, after Kubernetes.

AMAZON RELATIONAL DATABASE SERVICE

Amazon Relational Database Service (Amazon RDS) is a scalable relational database in the cloud. It provides cost-efficient and resizable capacity while automating time-consuming administration tasks such as hardware provisioning, database setup, patching and backups. It frees you to focus on your applications so you can give them fast performance, high availability, security and compatibility.

ANSIBLE

Ansible is a radically simple IT automation engine that automates cloud provisioning, configuration management, application deployment, intra-service orchestration, and many other IT needs. Designed for multi-tier deployments since day one, Ansible models your IT infrastructure by describing how all of your systems inter-relate, rather than just managing one system at a time.

PRIVACY

AWS SSO

AWS SSO manages all AWS accounts, cloud applications, AWS SSO-integrated applications, and custom SAML 2.0-based applications, without custom scripts or third-party SSO solutions.

TRACEABILITY

We use Prometheus and Sentry solutions for monitor, alert, and audit actions and changes to our environment in real time. Also Integrated log and metric collection with monitoring systems helm us to automatically investigate and take action

ANTIVIRUS

We use server-side virus scanners that can detect file uploads that contain malware, trojans, and viruses. The most common application for this task is ClamAV, an open-source antivirus engine.

VPN + SSO

AWS Client VPN is a managed client-based VPN service that enables us to securely access your AWS resources and resources in an on-premises network. With Client VPN, we can access our resources from any location using an OpenVPN-based VPN client. The VPN supports client authentication via SSO that allows setting flexible permissions compliant with our access policy

IAM

AWS Identity and Access Management (IAM) enables us to manage access to AWS services and resources securely. Using IAM, we can create and manage AWS roles and groups, and use permissions to allow and deny their access to AWS resources.

PASSWORD MANAGER

Standardizing under one centralized password management platform allows us to standardize and enforce password security policies across the organization, such as strong, unique passwords and using multi-factor authentication (2FA) on every site that supports it.

SECRETS MANAGEMENT

AWS Systems Manager, provides secure, hierarchical storage for configuration data management and secrets management. We store data such as passwords, database strings, etc.

2FA

2FA provides a second layer that ensures our systems are secure if one authentication factor is compromised

DATA PROTECTION (TRANSIT AND AT REST)

We classify our data into sensitivity levels and use mechanisms, such as encryption, tokenization, and access control where appropriate.

PASSWORD ROTATION

Password rotation is a universally accepted security best practice

SECURITY

1

CONTAINER SCANNING

Container Scanning performs vulnerability scans on container images in our Container Registry, it monitors the vulnerability information to keep it up to date. This process comprises two main tasks: scanning and continuous analysis.

2

DEPENDENCY SCANNING

The Dependency Scanning allows us to automatically find security vulnerabilities in our dependencies while we develop and test our applications. For example, dependency scanning lets us know if our application uses an external (open source) library that is known to be vulnerable. We can then take action to protect your application

3

CODE MANAGEMENT

(STATIC APPLICATION SECURITYTESTING)

We use automatic code review tool to detect bugs, vulnerabilities, and code smells in the code. We integrate it with our workflow to enable continuous code inspection across all projects

4

SECRETS ENCRYPTIONS

AWS Systems Manager, provides secure, hierarchical storage for configuration data management and secrets management. We store data such as passwords, database strings, etc.

AWS Key Management Service (KMS) allows administrators to create, delete and control keys that encrypt data stored in AWS databases and products.

5

SECURITY AUDITS

Successfully completed external audits from Breach Lock and ScienceSoft

breachlock





TECHNOLOGY

1

IAC

Infrastructure as Code (IaC) is an approach to managing data center server, storage, and networking infrastructure. IaC is meant to significantly simplify large-scale configuration and management. With traditional data center infrastructure management, every configuration change required manual action by operators and system administrators. With IaC, infrastructure configuration information is housed in standardized files, which can be read by software that maintains the state of the infrastructure. IaC improve productivity and reliability because it eliminates manual configuration steps.

2

AWS

(MULTI-REGION APPLICATION ARCHITECTURE SOLUTION ARCHITECTURE)

We use a Multi-Region Application Architecture to ensure resiliency of our applications. Back-end infrastructure deployed in the primary and secondary (failover) AWS Regions.

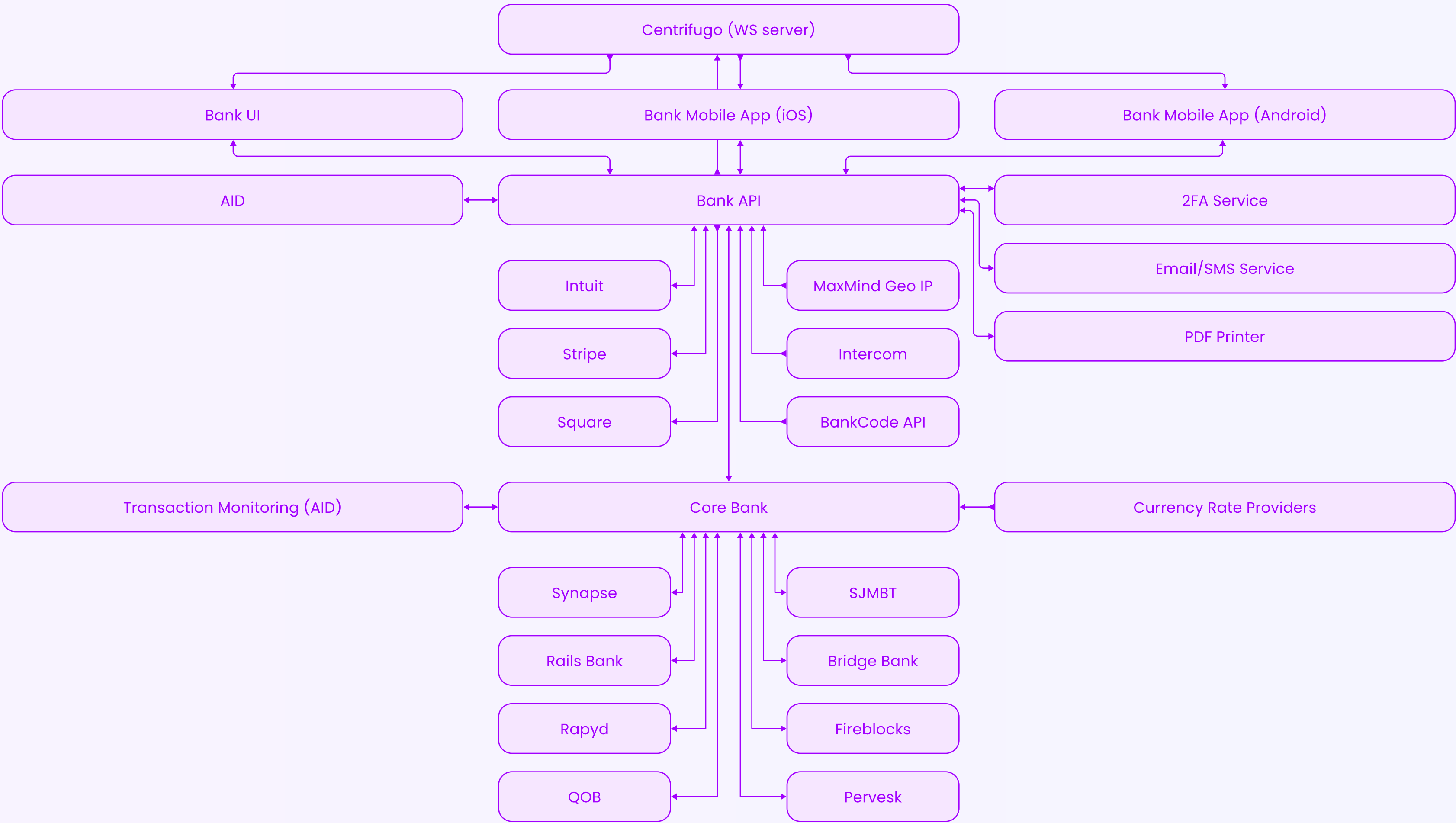
3

EKS

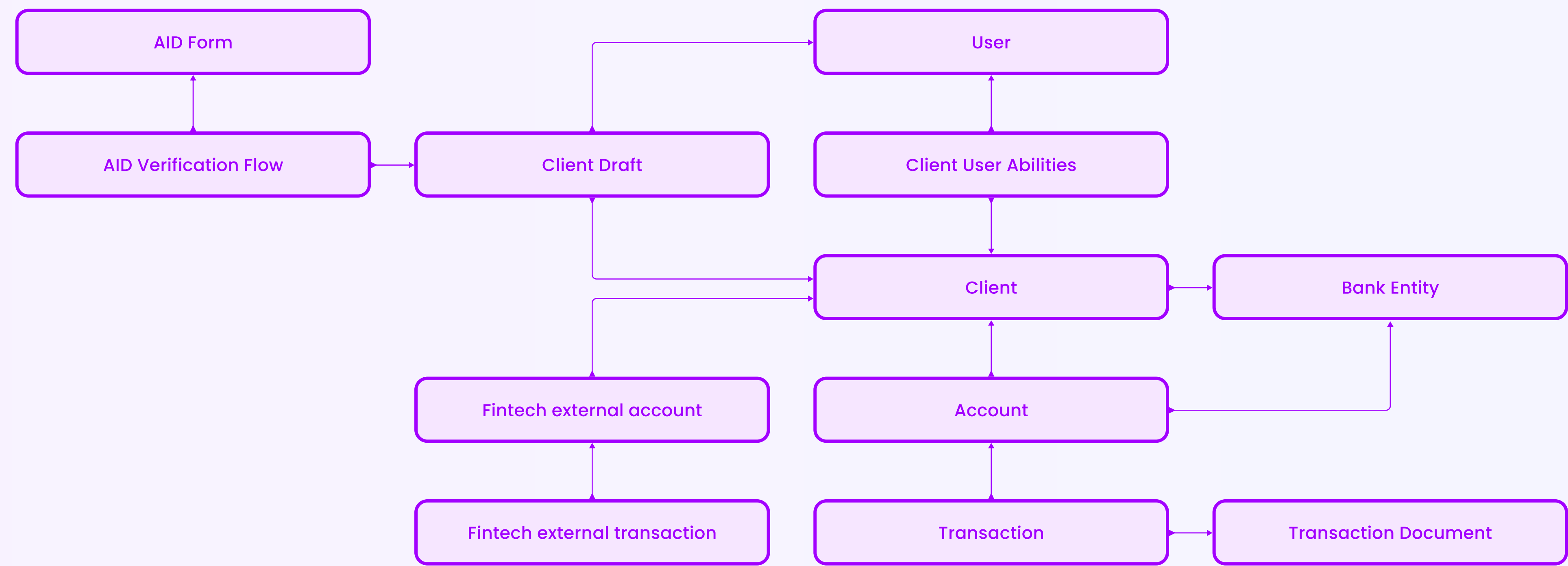
(KUBERNETES AND LOAD BALANCING)

We use kubernetes with load balancing tools. This allows us to save on our costs. The Kubernetes Cluster Autoscaler(CA) automatically adjusts the number of nodes in your cluster when pods fail or are rescheduled onto other nodes. The Cluster Autoscaler is typically installed as a Deployment in your cluster. It uses leader election to ensure high availability, but scaling is done by only one replica at a time. The Kubernetes Horizontal Pod Autoscaler(HPA) automatically scales the number of pods in a deployment, replication controller, or replica set based on that resource's CPU utilization. This can help your applications scale out to meet increased demand or scale in when resources are not needed, thus freeing up your nodes for other applications. When you set a target CPU utilization percentage, the Horizontal Pod Autoscaler scales your application in or out to try to meet that target.

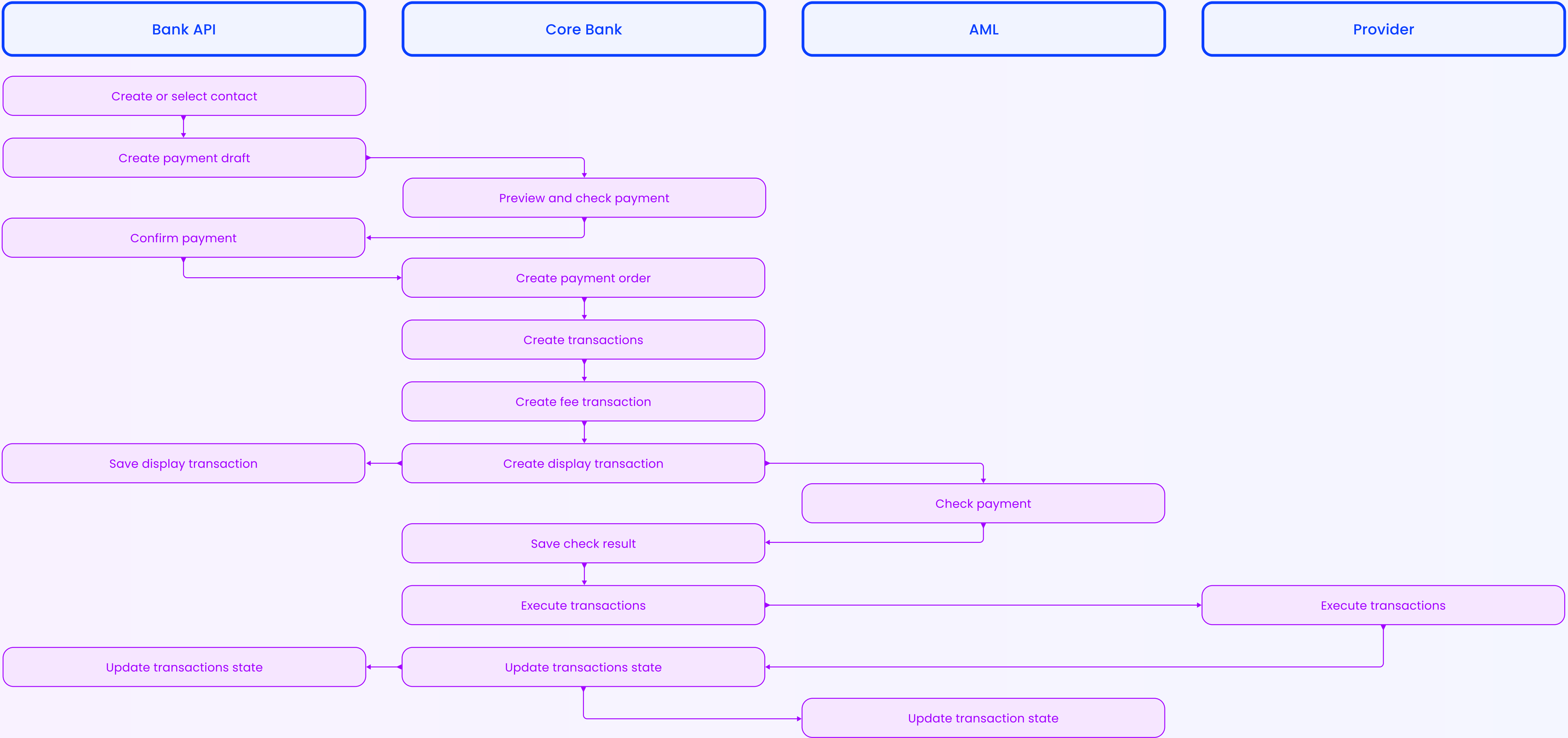
GENERAL ARCHITECTURE: BANK OVERVIEW DIAGRAM

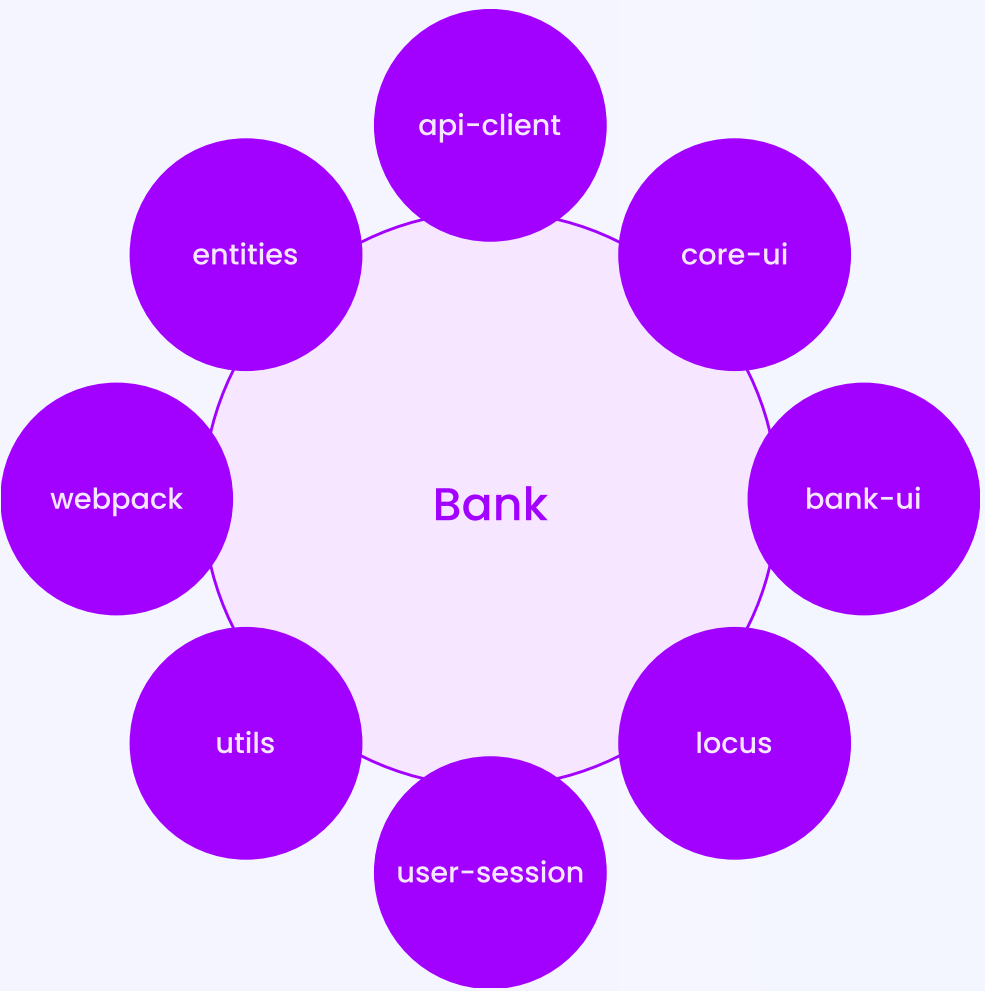
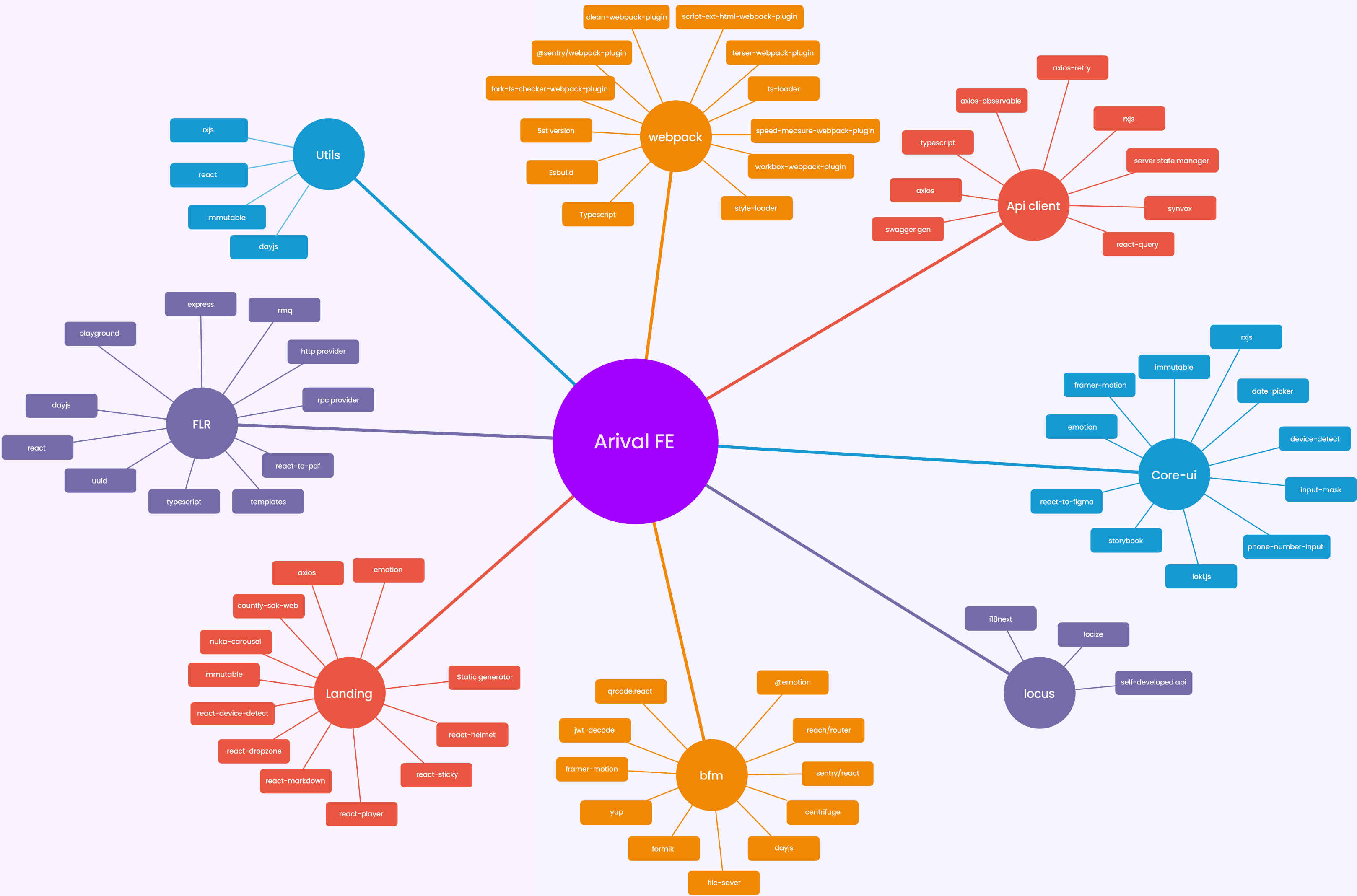


GENERAL ARCHITECTURE: SIMPLIFIED ER DIAGRAM



GENERAL ARCHITECTURE: PAYMENT DFD





Apps

- bfm
- landing
- flr-pdf-playground
- flr-pdf-service
- flr-email-playground
- flr-email-service

Packages

- api-client
- bank-landing-ui
- bank-ui
- bfm-ui
- locus
- user-session
- utils
- webpack
- pdf-templates
- email-templates

CORE ELEMENTS OF THE SYSTEM

AUTHORIZATION

- Jwt tokens (access + refresh)
- Automatic renewal when time runs out
- Tracking user activity and smart token refresh in the background
- Alerting the user when inactive and displaying the session timeout popup modal with blocking basic information

BACKOFFICE

Administrative tools to control bank data and payments.

CORE BANK

The service implements clients, accounts and transactions. It works with payment providers. It checks transactions with Arival compliance solution. It is master data storage and replicates data to Bank Api.

API CLIENT

We use Axios with the ability to get the monitored api client. On top of Axios, a server state manager is configured to reduce the number of requests to the backend (a kind of caching by key until the cache is invalidated)

BANK API

The service provides API for Web UI and for Mobile Applications. It works with users. It implements the onboarding process and user access management.

TECH NOTIFICATIONS

We also use a Centrifuge to update the current server state manager. With the help of notifications from the server-side, we timely update local data and display it in the user interface

DIFFERENT THEMES SUPPORT

We use styled-components (emotion) in conjunction with a dynamic set of css variables in which the palettes and color accents are protected. When changing a theme, there is no total re-renderer and page refresh due to the rejection of the built-in theme providers.

RENDERING

We are using polymorphic rendering, that is, partly on the server, partly on the client.

MULTI-COMPANIES

The interface has the ability to select the desired company and access only data and tools within the selected company based on the rights of this user

PRIVACY: ACCESS RIGHTS

The bank has flexible user permissions.
Each user can have access to several clients
and have different access to each account.
(One user can create the transaction,
other users can execute it)

- There are two modules within the core system.
- Access rights for each employee within the company
 - Access rights for each bank account

ABILITIES

cards	transfers	history	invoices	users_management	recipients	company
create	create internal	read	create	invite	read	read
	create external		read	update	modify	modify
				delete	add	

ACCOUNT ABILITIES

Selected account
view
create_draft
execute_draft

PRIVACY

abilities	
id	varchar
ability_group_id	uuid
name	string
value	string
created_at	datetime
updated_at	datetime

abilities_client_user_accounts	
id	varchar
client_user_account_id	uuid
ability_id	uuid

abilities_groups	
id	varchar
name	string
value	string
created_at	datetime
updated_at	datetime

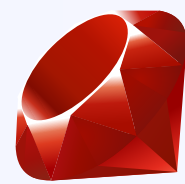
client_users	
id	varchar
client_id	uuid
state	string
aid_customer_id	string
aid_case_id	string
aid_profile_id	string
rejected reason	string
is_new	boolean
position	string
activated_at	datetime
role	string

abilities_client_users	
id	varchar
client_user_account_id	uuid
ability_id	uuid

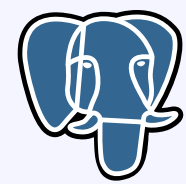
SECURITY

- CSP is configured across all client applications. We update dependencies in a timely manner and discard unnecessary ones during refactoring.
- We store tokens at the current moment in local storage. Implemented technical solutions for token encryption with a pin code and secure storage.
- Compliant and Improved according to ASVS L2
- The source code is minified and obfuscated during the build, comments are removed. Configured CORS on the server-side.
- On the client-side, an error boundary mechanism is built to handle errors. All errors are sent to self-hosted Sentry.
- We automatically run a yarn audit on every development change. Wapiti starts up periodically and each time before production release.
- The bank database stores only encrypted user’s data. It is implemented with 256-bit AES encryption. Search indexes are created using Argon 2 hashing algorithm (can not be brute forced).
- The bank has a strong TFA subsystem. It supports verification through SMS, Voice Cal, or TOTP application.
- Each user login is saved with device info and geolocation. Users are informed about all suspicious login attempts (after it was blocked).
- Multilayer architecture protects the bank from various attacks. Only bank API is directly accessible from the Internet. The Core bank and AML services work with Bank API though secured RPC-api in the closed internal network.

TECHNOLOGY: BANK KEY TECHNOLOGIES



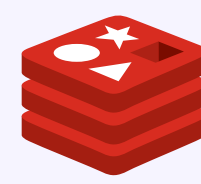
Ruby



Postgres



AWS



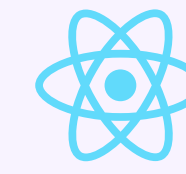
Redis



RabbitMQ



Go lang



React



TypeScript

Bank uses

- **Sinatra framework** – a very lightweight and fast API implementation.
- **Sidekiq** – a powerful background job scheduler.
- **Trailblazer** – a structured business logic framework.
- **ActiveRecord** and **Active Support** – a brilliant part of Ruby on Rails.



ARRIVAL ONBOARDING

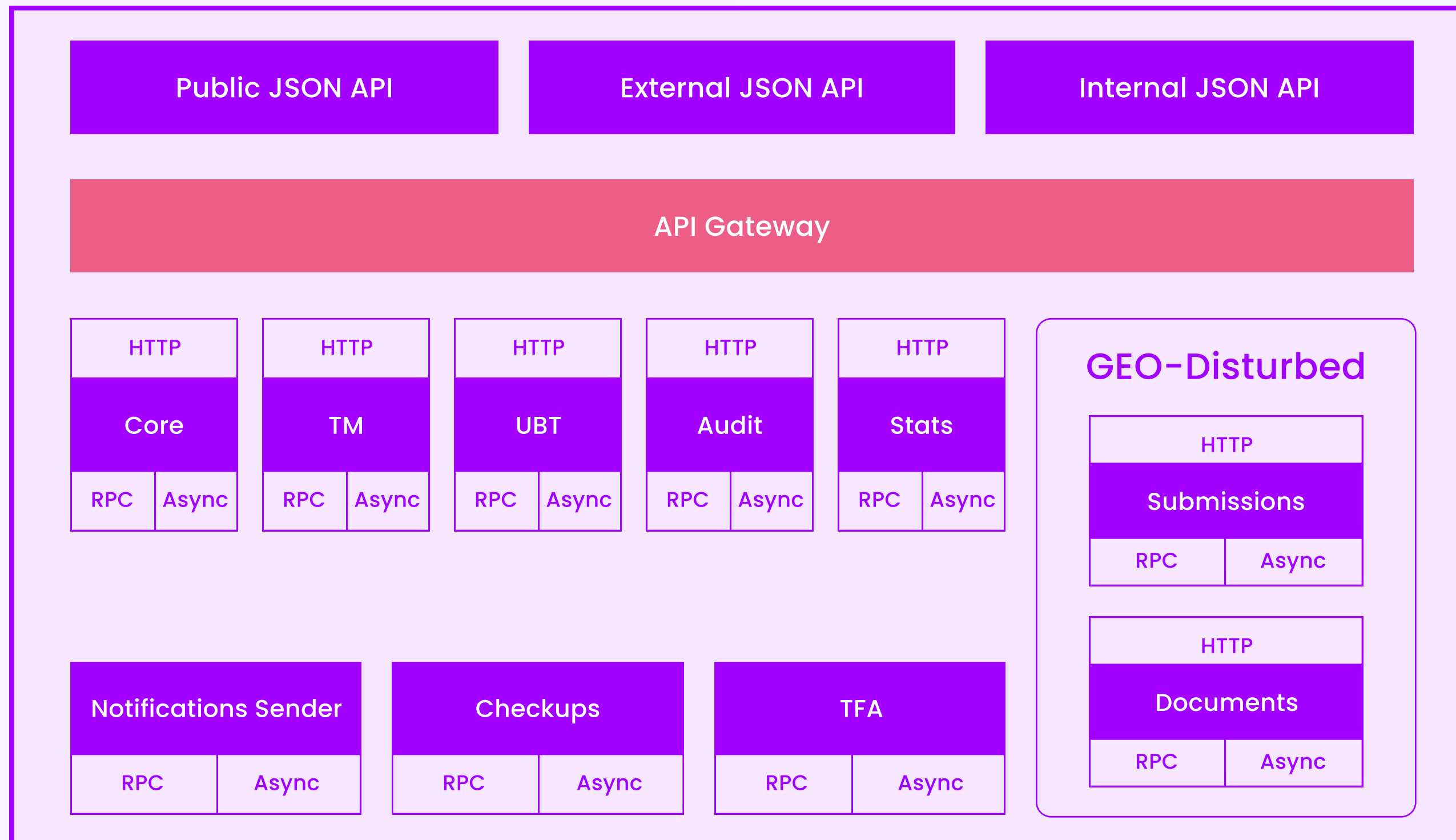
And compliance solution (A.ID)



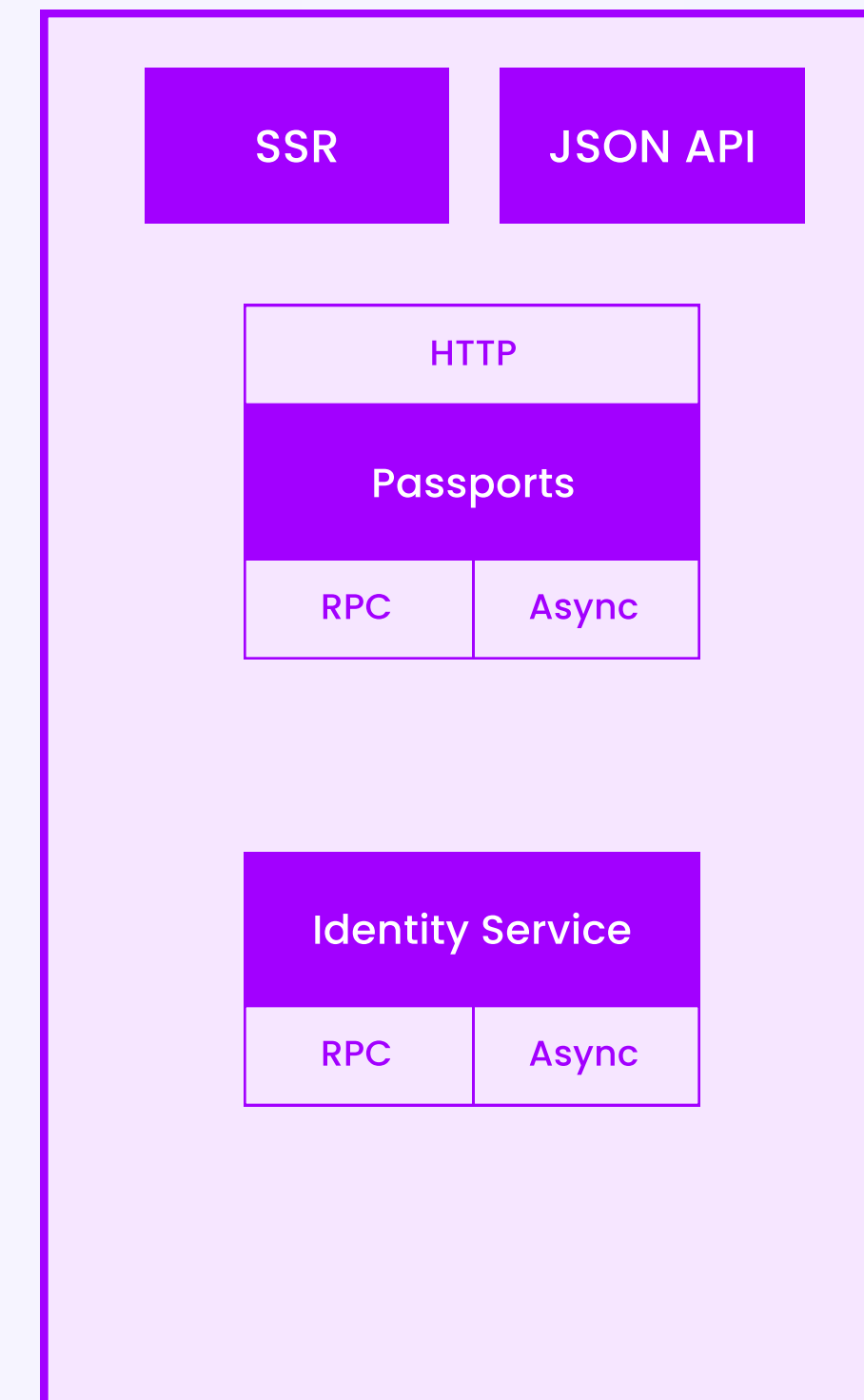


GENERAL ARCHITECTURE: BACKEND

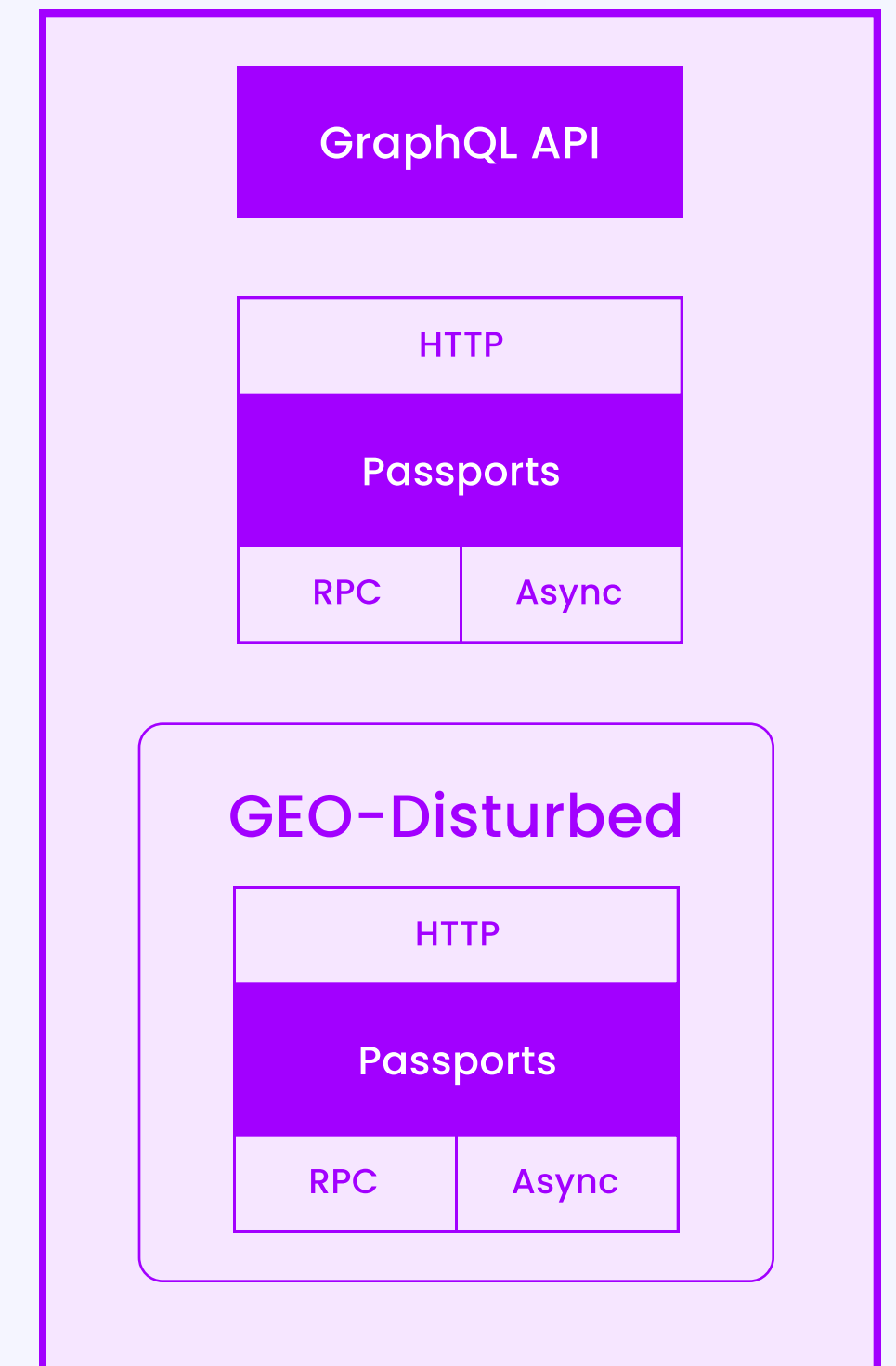
CORE



AUTH



PASSPORT



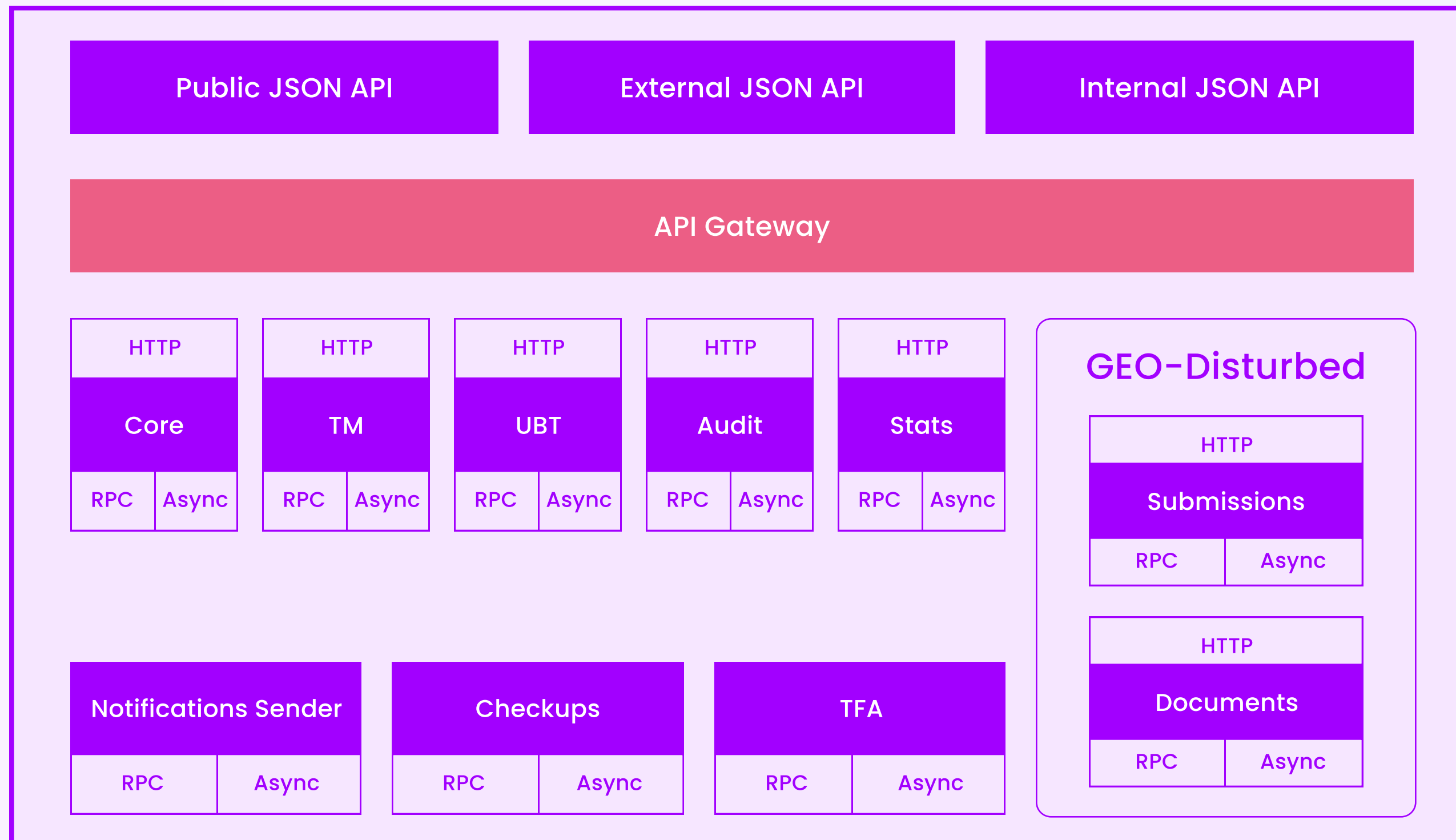
System Bus



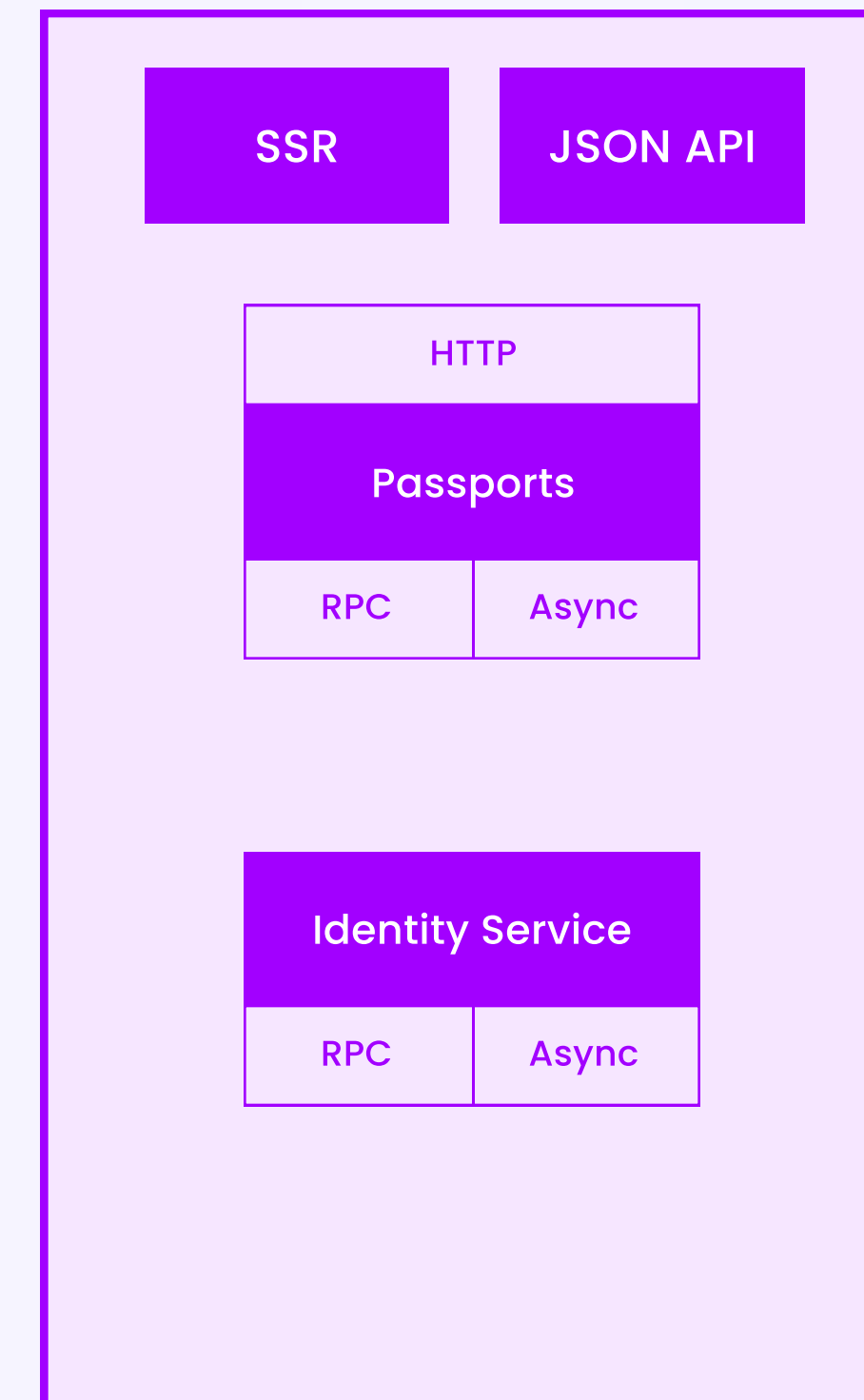


GENERAL ARCHITECTURE: BACKEND

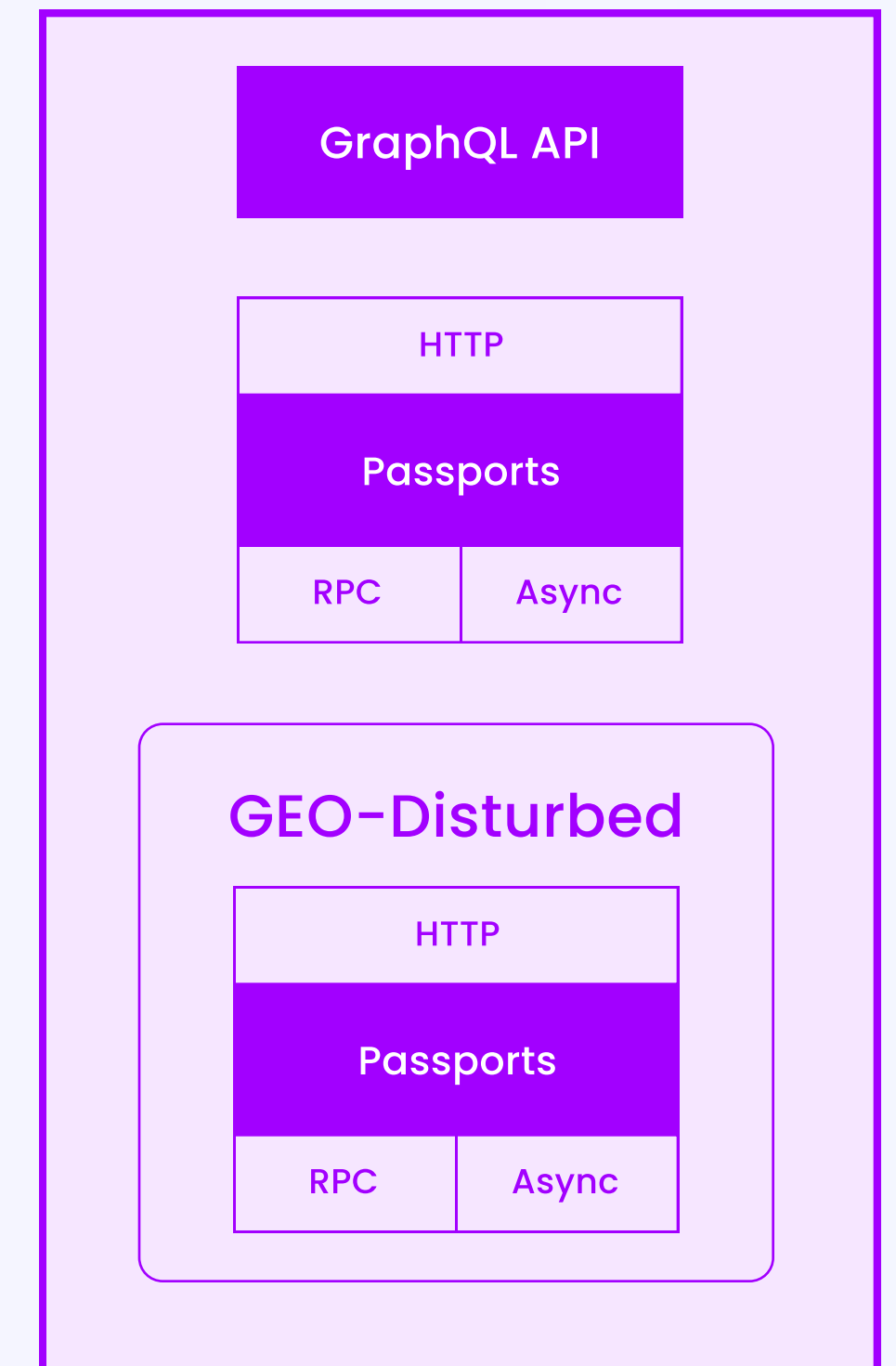
CORE



AUTH



PASSPORT

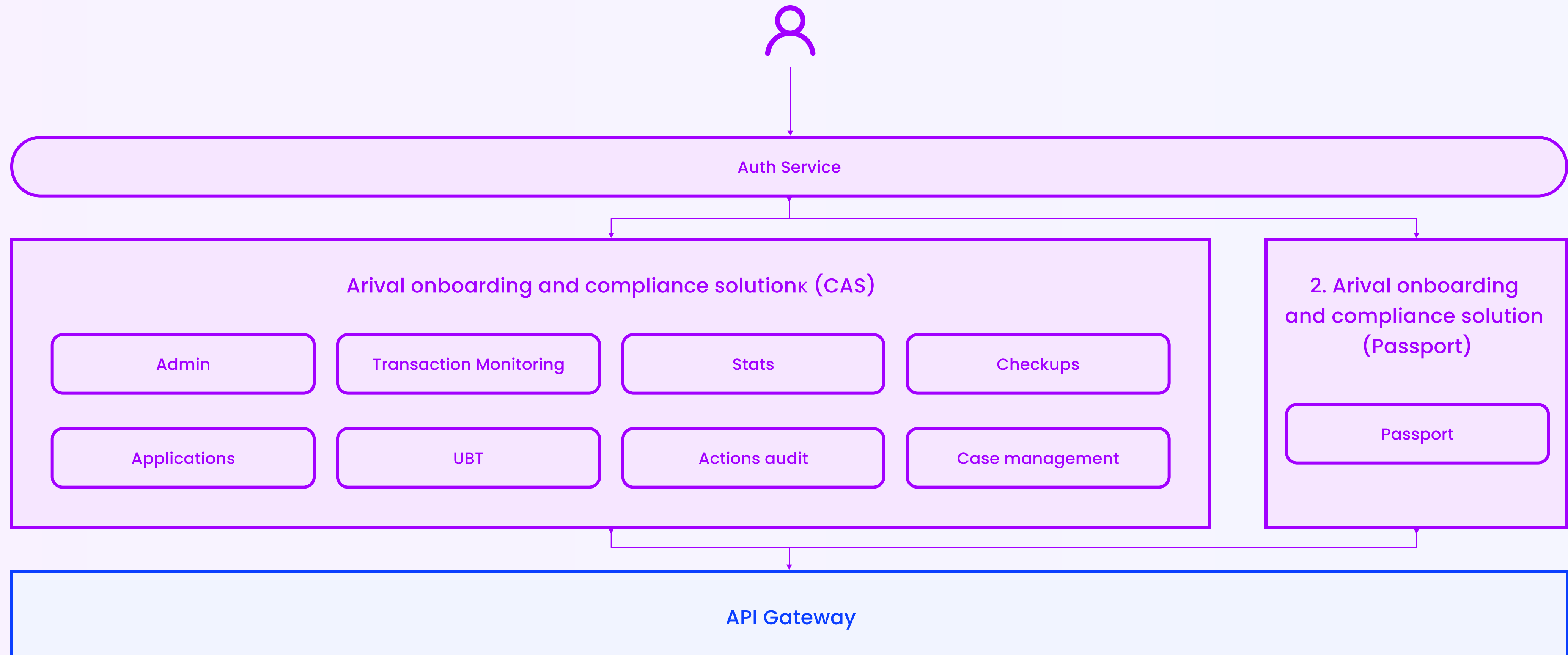


System Bus





GENERAL ARCHITECTURE: FRONTEND





SERVICES

ARRIVAL ONBOARDING AND COMPLIANCE SOLUTION COMPLIANCE SUITE

- 1. Core.** Main Compliance Suite module.
 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 2. Transaction Monitoring.**

Module for monitoring and checking financial transactions.

 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 3. User Behaviour Tracking.**

Tracks user's action performed in A.ID UI.

 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 4. Actions Audit.**

Collects all the business actions performed by the user.

 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)

ARRIVAL ONBOARDING AND COMPLIANCE SOLUTION COMPLIANCE SUITE

- 5. Stats.** Provides aggregation and calculation of various statistical data.
 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 6. Submissions.** Stores users applications for forms. Geo-distributed service.
 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 7. Checkups.** Performs checkups for audit.
 - a. PL: Ruby
 - b. DB: PostgreSQL (RDS)

ARRIVAL ONBOARDING AND COMPLIANCE SOLUTION PASSPORT

- 1. Passports.** Proxy service for accessing passports data.
 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 2. Passport Data.** Service for storing passports data.
 - a. PL: Ruby / GraphQL Ruby
 - b. DB: PostgreSQL (RDS)

SHARED

- 1. Auth.** SSO service with OAuth2 framework implementation.
 - a. PL: Golang
 - b. DB: PostgreSQL (RDS)
- 2. Identity.** Service for storing user's passwords.
 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 3. Documents.** Service for storing documents.

Supports publicly accessible and private documents.

 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 4. Notifications Sender.**

Sends notifications via email, sms, by robot calls etc.

 - a. PL/Framework: Ruby / Sinatra
 - b. DB: PostgreSQL (RDS)
- 5. 2FA-service.** Implements two-factor authentication.
 - a. PL: Golang
- 6. Link shortener (Shorty).** Shortifies links.
 - a. PL: Golang





PRIVACY & SECURITY

HOW WE MAINTAIN ARIVAL ONBOARDING AND COMPLIANCE SOLUTION DATA PRIVACY & SECURITY:

BACKEND

- Secure passwords storing (hash + salt)
- No passwords stored in publicly accessible services
- Mandatory Two-Factor Authentication
- User sensitive data stored in Geo-distributed services (dependent on user region).
- User sensitive data is encrypted in databases.
- Every service-to-service communication is encrypted.
- User is able to track who he shared his data with.
- Strong access granting and permissions model.

FRONTEND

- Default XSS Protection with Data Binding
- Dangerous URLs
- Rendering HTML
- Direct DOM Access
- Server-side Rendering
- Detecting Vulnerabilities in Dependencies
- Injecting JSON State
- Detecting Vulnerable Versions of React
- Configuring Security Linters
- Avoiding Dangerous Library Code





TECH STACK

BACKEND

1. PL/Frameworks:

- a. Ruby / Sinatra
- b. Golang
- c. JavaScript(TypeScript) / React

2. Infra:

- a. AWS
- b. K8S
- c. Docker

3. Storage:

- a. PostgreSQL (RDS)
- b. Redis

4. Storage: RabbitMQ

FRONTEND

1. Arival onboarding and compliance solution (CAS)

- a. JavaScript (TypeScript)
- b. ReactJS

2. Arival onboarding and compliance solution (Passport)

- a. JavaScript (TypeScript)
- b. ReactJS

